

Security Audit

Allfeat (DAO)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Audit Findings	12
Centralisation	18
Conclusion	19
Our Methodology	20
Disclaimers	22
About Hashlock	23

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Allfeat team partnered with Hashlock to conduct a security audit of their main repository that contains the pallets, the runtime and the node. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Allfeat is a pioneering Layer 1 blockchain platform tailored specifically for the music industry, seamlessly integrating Web3 technology to revolutionize how music metadata is secured and monetized. By leveraging Music Industry Decentralized Data Structures (MIDDS), Allfeat ensures transparent and immutable management of artist profiles, tracks, and royalties, fostering trust and efficiency among creators, distributors, and listeners. Built on a robust Substrate/Rust core, it employs a unique Proof of Metadata consensus mechanism, empowering community participation in data certification and governance through a decentralized autonomous organization (DAO). With multi-chain oracle capabilities, Allfeat provides reliable, cross-chain data access for decentralized applications (dApps), positioning itself as the definitive Web3 infrastructure for a fair and interconnected music ecosystem.

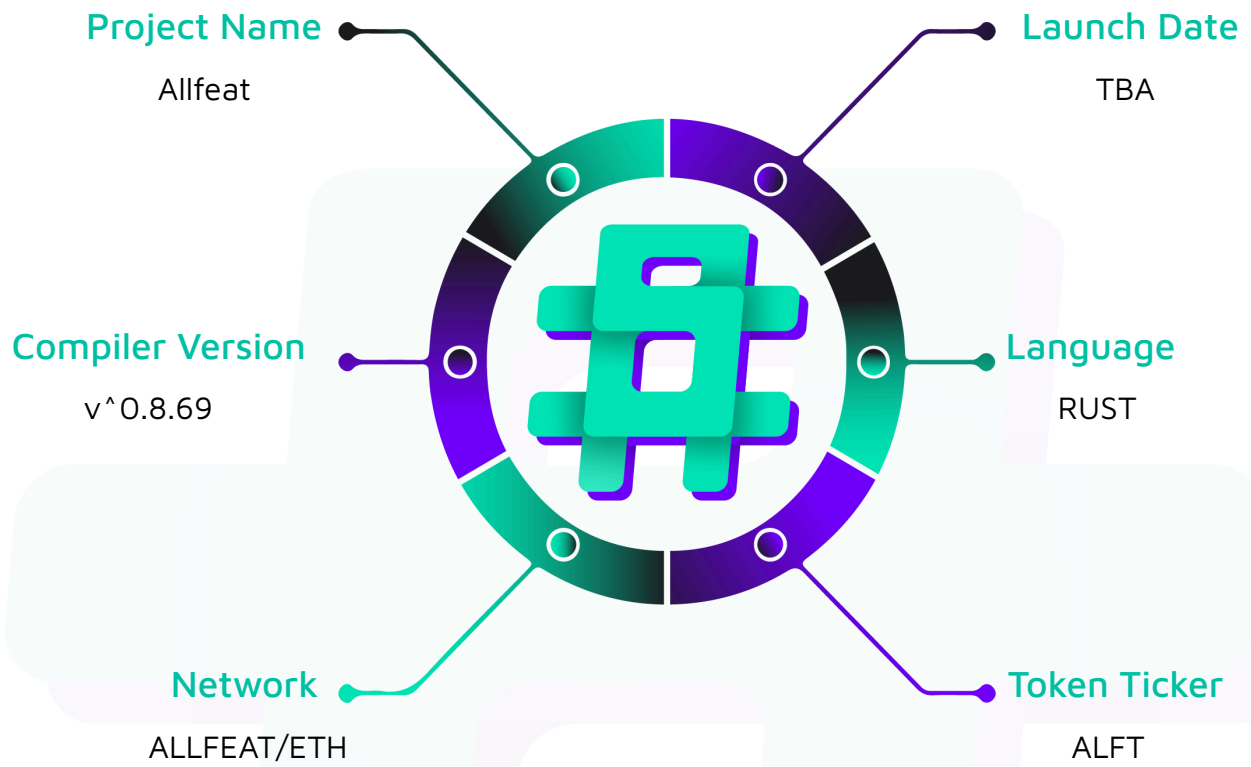
Project Name: Allfeat

Compiler Version: ^0.8.42

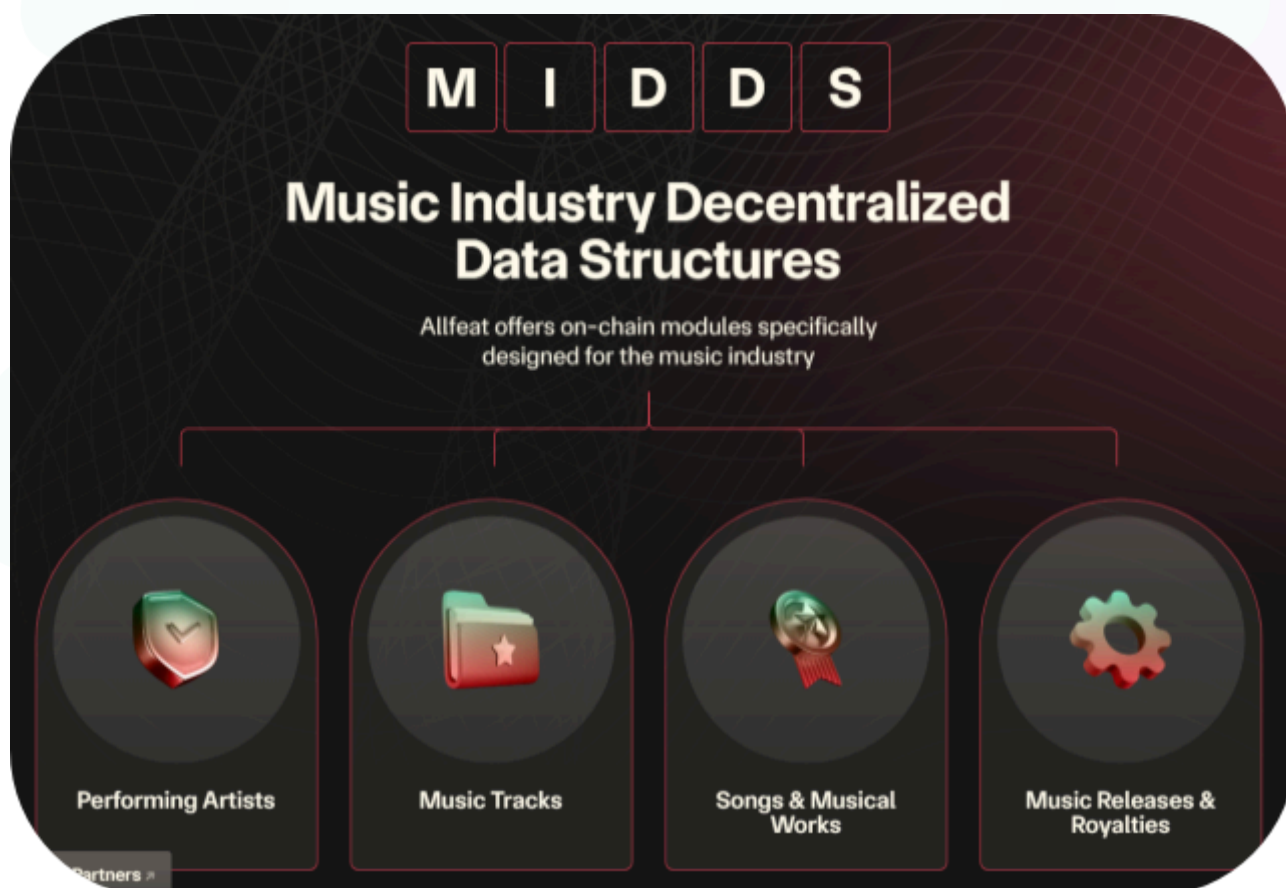
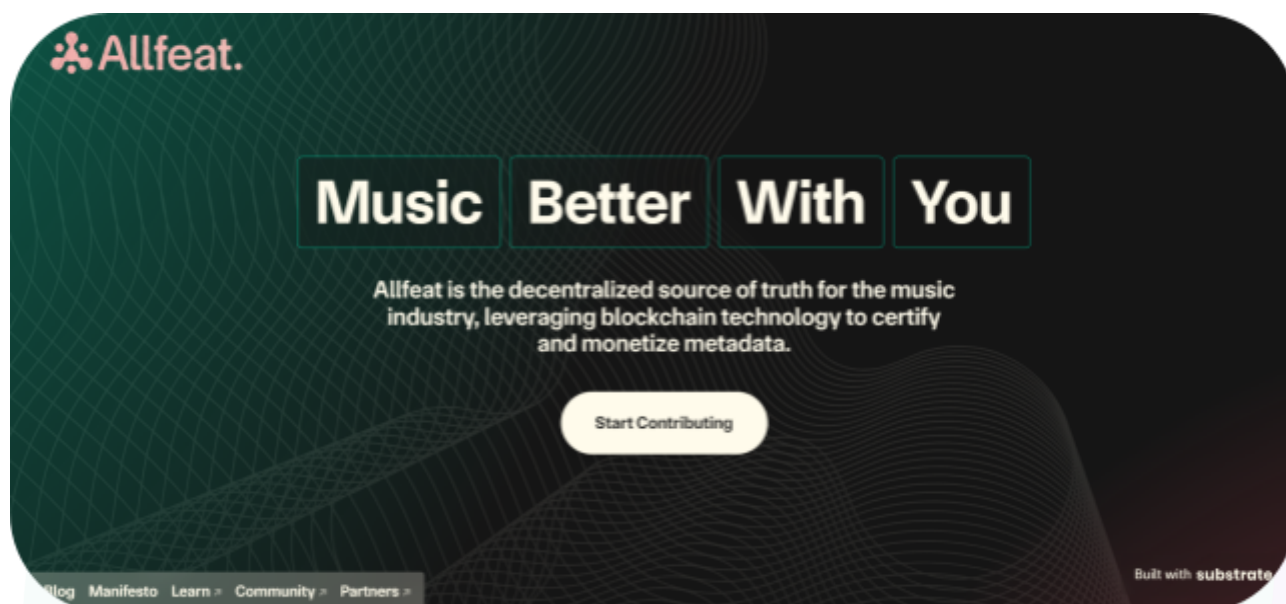
Website: <https://www.allfeat.com/>

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the Rust code within the Allfeat main repository, the scope of work included a comprehensive review of the smart pallets and other files listed below. We tested the protocol's pallets and runtime to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Allfeat Protocol Runtime and Node
Platform	Allfeat blockchain (Substrate-based), Rust
Audit Date	December, 2024
Main repository	Allfeat
GitHub Commit Hash	https://github.com/Allfeat/Allfeat

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Parity contracts. We initially identified some significant vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 Medium severity vulnerabilities

2 Low severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Pallets - Allows users to: - Register midds - Supply the collateral according to the midds bytes size - Deregister midds - Withdraw their initial collateral - Update fields of the pending midds	Contract achieves this functionality.
Runtime and node - This block specifies different parameters of the network	Protocol achieves this functionality.

Code Quality

This audit scope involves the pallets, the runtime and the node of the Allfeat protocol, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Allfeat code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Audit Findings

Medium

[M-01] lib.rs#calculate_mids_colateral() - Lack of deposit bytes limit would allow storing free bytes after the maximum value is reached

Description

In the current implementation of the pallets, providers have to pay a deposit cost for each byte deposited. However, there is no cost for them if the maximum value of the balance type is reached. The lack of bytes limit allows attackers to specify any byte size and they will only be charged only with the maximum value of the certain type.

Vulnerability Details

The current functionality of `lib.rs` allows providers to deposit any number of bytes and pay the corresponding deposit cost:

```
fn calculate_mids_colateral(mids: &MidsWrapperOf<T, I>) -> BalanceOf<T, I> {  
  
    let bytes = mids.mids.total_bytes();  
  
    T::ByteDepositCost::get().saturating_mul(BalanceOf:::<T, I>::from(bytes))  
  
}
```

`saturating_mul()` method is used as an overflow protection measure but it will cap the value of the balance type at its maximum to prevent overflow. However, if the multiplication result of the deposited bytes and the deposit cost exceed the balance type maximum value (like `u64` or `u128`), providers would not pay any cost above that while having their bytes stored.

Impact

Providers will not be charged any cost upon the maximum value of the balance type. Although this is quite an extreme condition, the protocol has to work expectedly under

any circumstances. Malicious providers with a huge amount of native token holdings can exploit this to put some inadequate amount of bytes to be stored in the system.

Recommendation

Introduce a limit on the number of bytes that can be deposited along with the mids.

Status

Resolved

[M-02] runtime#balances.rs - Low existential deposit creates a possibility of the storage bloating attack

Description

In the current implementation of the Allfeat protocol, an existential deposit is represented in terms of the native protocol token that's not tradable yet. This creates a possibility of a storage bloating attack. At the moment it's set to the following value:

Vulnerability Details

On Substrate-based runtimes, the Existential Deposit (ED) is the minimum balance that is needed to have an active account (similar to the rent on Solana). This is particularly useful to prevent the creation of dust accounts and bloating of the storage. In this case, if the balance of an account falls below ED, the account will be removed from the storage and the nonce will be reset.

Storage usage has cost in terms of fees which impacts all the users, as there is a bigger overhead. In the Allfeat protocol, the ED is set to 1 MILLIALFT:

```
parameter_types! {  
    pub const ExistentialDeposit: Balance = MILLIALFT;  
  
}
```

To estimate an attack from the price perspective, let's consider the ED on the Polkadot network. The ED parameter on Polkadot is 1 DOT. So to create 1 million accounts, a user

would have to pay about 8 million dollars (to be distributed as dust among accounts). However, in the case of Allfeat, the token is not tradeable yet and this introduces a risk of a storage attack. For instance, if the ALFT token has a price of 0.002\$, the cost of creating 1 billion accounts would be just 2000\$ which is quite low.

```
pub const MICROALFT: Balance = 1_000_000;  
  
pub const MILLIALFT: Balance = 1_000_000_000;  
  
pub const ALFT: Balance = 1_000_000_000_000;
```

Impact

The storage can be bloated with dust accounts at a low cost.

Recommendation

Consider setting the ED parameter according to the expected price of the token or at least at 1 ALFT similar to how DOT (the biggest unit) is set on the Polkadot network.

Status

Resolved

Low

[L-01] lib.rs#unregister() - Reliance on block number instead of timestamp

Description

Currently, the Allfeat protocol uses `block.number` as a time unit instead of using `block.timestamp`:

```
let now = <frame_system::Pallet<T>>::block_number();

        let spent = now - mids.registered_at();

        ensure!(

            spent

                >

polkadot_sdk::sp_runtime::SaturatedConversion::saturated_into(

                    T::UnregisterPeriod::get()

                ),

            Error::<T, I>::UnregisterLocked

        );
```

The fact that some chains are subjected to reorgs introduces an additional risk where providers can be able to unregister (or, vice versa, not able) due to the chain reorganization.

Recommendation

Use `block.timestamp` instead of `block.number`

Status

Resolved

[L-02] lib.rs#register() - Front-running attacks are possible to be performed on the providers when registering

Description

In the Allfeat protocol, providers are entities that register their midds and pay the corresponding byte deposit cost for each byte deposited. The problem is that the registration mechanism is subjected to front-running attacks where malicious users can front-run a provider and register with the same midds creating a pending midds that will not allow a legit provider to register their own ones.

Vulnerability Details

According to the current implementation of the midds pallet, the function that's needed to be called for midds registration is `register()`:

```
#[pallet::call_index(0)]

pub fn register(origin: OriginFor<T>, midds: Box<T::MIDD>) ->
DispatchResult {

    let provider = T::ProviderOrigin::ensure_origin(origin)?;

    let midds =

        MiddsWrapper::new(provider,
<frame_system::Pallet<T>>::block_number(), *midds);

    Self::inner_register(midds)

}
```

The provider is automatically set to the `caller` and then the midds are registered in the internal function with this caller:

```
let midds_hash = midds.midds.hash();

// Verify that the same MIDD hash isn't registered already.

ensure!(

    !PendingMidds::<T, I>::contains_key(midds_hash),
```



```
Error::<T, I>::MiddsDataAlreadyExist  
);
```

As you can see here, first, the function calculates `midds_hash` if it becomes included in the `PendingMidds`, it's no longer possible to register midds with the same hash.

Impact

Providers will not be able to register the same midds if the registration transaction was front-run. This is especially crucial if the bytes deposit cost is not very huge so the attack becomes easily reproducible.

Recommendation

Consider introducing additional fee parameter that's taken on registration or creating a whitelist including the providers that can register their midds. The first solution is more optimal as it keeps the decentralization aspect of the protocol.

Note

The Allfeat team informed Hashlock that the goal is to have as many valid midds on the chain as possible resulting the conflict between data not being an issue to concern.

Status

Acknowledged

Centralisation

The Allfeat project values community-centered and transparency over centralisation.

Allfeat emphasizes a community-centric approach while maintaining transparency. Although some critical functions remain centralized for enhanced security and functionality, the project strives to balance decentralization with internal team accountability. This blend ensures operational security while keeping the protocol aligned with decentralized values.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Allfeat project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.